

SRINIVAS UNIVERSITY
INSTITUTE OF ENGINEERING & TECHNOLOGY



(SUBJECT: ADVANCED MACHINE LEARNING)
(SUBJECT CODE: 24SAL051)

AN INDIVIDUAL TASK REPORT ON

“Loan Approval Prediction Using Machine Learning Algorithms”

Submitted in the partial fulfilment of the requirements for the
Fifth semester

BACHELOR OF TECHNOLOGY
IN
Artificial Intelligence & Machine Learning

Submitted By,

Chethan.R
01SU24AI022

UNDER THE GUIDANCE OF
Prof. Mahesh Kumar V B

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE
LEARNING
SRINIVAS UNIVERSITY INSTITUTE OF ENGINEERING &
TECHNOLOGY
MUKKA, MANGALURU-574146

2026-27

SRINIVAS UNIVERSITY
INSTITUTE OF ENGINEERING & TECHNOLOGY

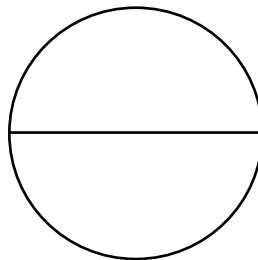


Department of Artificial Intelligence & Machine Learning

CERTIFICATE

This is to certify that, **CHETHAN. R (01SU24AI022)**, has satisfactorily completed the assessment (Individual Task Report) in **Advanced Machine Learning (24SAL051)** prescribed by the Srinivas University for the 5th semester B. Tech. course during the year **2026-27**.

MARKS AWARDED



Staff In charge

Prof. Mahesh Kumar V B

TABLE OF CONTENTS

No.	Section	Page
1.	Introduction	1
2.	Problem Statement, Objectives and Scope	3
3.	Dataset and Data Understanding	5
4.	System Requirements and Development Environment	7
5.	Methodology and System Architecture	8
6.	Machine Learning Models	9
7.	Implementation and Experimental Procedure	11
8.	Results and Model Comparison	12
9.	Discussion, Limitations and Applications	14
10.	Conclusion, Future Scope and References	15

ABSTRACT

Loan approval is an important decision in financial services because it determines whether an applicant receives the requested credit. Manual assessment can be time-consuming and may be affected by inconsistent evaluation of applicant information. Machine learning provides a practical way to learn patterns from historical loan records and predict whether a loan application is likely to be approved or rejected.

This project develops a supervised machine learning classification system for Loan Approval Prediction. The dataset contains 1,150 records and 10 attributes covering applicant characteristics, income, loan amount, credit history, property area and loan status. The target variable, Loan_Status, is converted into two classes: Approved and Rejected. Categorical attributes are encoded into numerical values, missing values are handled using median-based filling where required, and the data is divided into training and testing sets using an 80:20 stratified split.

Three classification algorithms are evaluated: Gaussian Naive Bayes, Decision Tree and Logistic Regression. Standardization is applied to the data used by Naive Bayes and Logistic Regression, while the Decision Tree is trained on the non-standardized feature values. The Decision Tree uses the Gini criterion and a maximum depth of four, making its structure easier to interpret.

The models are compared using accuracy, precision, recall and F1-score. On the evaluated dataset, the Decision Tree achieved the highest accuracy of 93.04%, followed by Naive Bayes at 88.70% and Logistic Regression at 87.83%. The Decision Tree is therefore selected as the best-performing model for this experimental setup.

The proposed system can assist financial institutions in making faster and more consistent loan decisions by reducing the need for completely manual evaluation of applications. It can help identify important applicant factors that influence loan approval outcomes and support efficient decision-making. The developed approach demonstrates the practical application of machine learning to real-world financial problems.

Keywords

Machine Learning, Loan Approval, Classification, Naive Bayes, Decision Tree, Logistic Regression, Credit History, Predictive Analytics

1. INTRODUCTION

1.1 Background

Financial institutions receive large numbers of loan applications containing information such as income, loan amount, credit history, employment-related attributes and property details. The final decision generally belongs to one of two categories: approval or rejection. Predictive analytics can assist this process by learning relationships between historical application information and previously recorded decisions.

1.2 Motivation

The motivation for this project is to demonstrate how machine learning can be used for a practical binary classification problem. Loan approval prediction is suitable for comparing algorithms because the input contains both numerical and categorical information and the output is a clearly defined class.

1.3 Need for Algorithm Comparison

Different algorithms make different assumptions about the data. Naive Bayes is probabilistic and computationally lightweight. Logistic Regression provides a linear decision boundary and is widely used for binary classification. Decision Tree can learn non-linear decision rules and is easy to interpret. Comparing all three on the same training and test split gives a more meaningful evaluation than relying on a single algorithm.

1.4 Project Overview

The workflow consists of dataset loading, column-name cleaning, categorical encoding, feature-target separation, missing-value handling, train-test splitting, feature standardization where appropriate, model training, prediction and evaluation. The final stage compares the models and selects the algorithm with the highest test accuracy.

1.5 Machine Learning in Diabetes Prediction

A machine learning loan classifier does not replace the complete financial decision process. Instead, it provides a data-driven prediction based on the patterns represented in the training data. Such a system can be useful for academic demonstrations, preliminary screening and decision-support applications, provided that fairness, privacy, regulation and human oversight are considered

1.6 Advantages of the Proposed Approach

The proposed approach provides the following advantages:

- Uses historical data to learn patterns associated with loan approval.
- Provides a repeatable and measurable classification procedure.
- Allows multiple algorithms to be compared on the same dataset.
- Uses standard evaluation metrics rather than accuracy alone.
- Provides an interpretable Decision Tree model for understanding decision rules.

1.7 Report Organization

The report first defines the problem and objectives, then explains the dataset and development environment. The methodology and algorithms are described next, followed by implementation, experimental results, model comparison, discussion, limitations, applications, conclusion and future scope.

2. PROBLEM STATEMENT, OBJECTIVES AND SCOPE

2.1 Problem Statement

To develop a machine learning based classification system that predicts whether a loan application is likely to be Approved or Rejected using applicant and loan-related attributes, and to compare the performance of Naive Bayes, Decision Tree and Logistic Regression on the same dataset.

2.2 Objectives

The main objectives of the project are:

- To understand and preprocess a structured loan approval dataset.
- To identify relevant numerical and categorical features for classification.
- To encode categorical attributes into numerical representations.
- To divide the dataset into training and testing subsets using an 80:20 split.

- To standardize features for models that benefit from comparable feature scales.
- To train Gaussian Naive Bayes, Decision Tree and Logistic Regression classifiers.
- To evaluate the models using accuracy, precision, recall and F1-score.
- To analyze the Naive Bayes confusion matrix and Decision Tree structure.

To compare the algorithms and identify the best-performing model

2.3 Scope

The scope of this project is limited to the experimental classification of the Scikit-learn diabetes dataset.

The project covers:

- Dataset loading
- DataFrame creation
- Target transformation
- Feature-target separation
- Train-test splitting
- Feature scaling
- Classification model training
- Prediction
- Performance evaluation
- Visualization
- Algorithm comparison

The project does not represent a clinical diagnosis system. The two outcome classes are generated specifically for this machine learning experiment.

2.4 Expected Outcomes

The expected outcomes of the project are:

- Successful loading and processing of the dataset.
- Creation of two project-defined outcome classes.
- Successful training of three classification algorithms.
- Calculation of classification performance metrics.
- Generation of visualizations.
- Identification of the best-performing algorithm based on F1 Score.

3. DATASET AND DATA UNDERSTANDING

3.1 Dataset Description

The dataset used for the experiment contains 1,150 rows and 10 columns. It represents loan applications and includes applicant information, financial attributes, credit history, property information and the final loan decision. The target column is Loan_Status.

Feature	Type	Description
Gender	Categorical	Applicant gender
Married	Categorical	Marital status
Education	Categorical	Graduate / Not Graduate
ApplicantIncome	Numerical	Applicant income
CoapplicantIncome	Numerical	Co-applicant income
LoanAmount	Numerical	Requested loan amount
Credit_History	Numerical	Credit history indicator
Property_Area	Categorical	Rural / Semiurban / Urban
Loan_Term	Numerical	Loan repayment term
Loan_Status	Target	Approved / Rejected

3.2 Feature and Target Separation

The target column Loan_Status is separated from the input features. The remaining columns form the feature matrix X, while the encoded loan decision forms the target vector y.

3.3 Important Implementation Note

The available 1,150-row dataset contains the ten fields listed above. Therefore, the categorical preprocessing used for the reported experiment is aligned with these actual columns. In particular, the dataset does not contain a `self_employed` column or a `loan_id` column, so those references in the supplied code should be removed or adjusted when running the code with this dataset.

The available 1,150-row dataset contains the ten fields specified for the project, and the preprocessing steps are therefore designed according to these actual columns. Categorical attributes present in the dataset are converted into numerical values so that they can be processed effectively by the machine learning algorithms. Missing values are also handled appropriately to ensure that the dataset is suitable for training and testing the models.

It is important to note that the dataset does not contain the **`self_employed`** or **`loan_id`** columns mentioned in the originally supplied code. Therefore, these references should be removed or modified when implementing the project with the current dataset. This ensures that the preprocessing and model training steps match the actual dataset structure and prevents errors during execution.

3.4 Data Preprocessing

The preprocessing procedure consists of the following steps:

1. Load the dataset.
2. Convert the dataset into a Pandas DataFrame.
3. Create the Outcome column.
4. Separate features and target.
5. Split the data into training and testing sets.
6. Apply StandardScaler to the data used by Logistic Regression and Naive Bayes.

The code checks the dataset shape and first few records using:

```
print("Dataset Shape:", df.shape)

print(df.head())
```

3.5 Feature and Target Separation

The input features are separated from the Outcome column:

```
X = df.drop("Outcome", axis=1)
```

```
y = df["Outcome"]
```

Here:

- X contains the ten input features.
- y contains the binary Outcome values.

The feature matrix is used as input to the machine learning models.

4.SYSTEM REQUIREMENTS AND DEVELOPMENT ENVIRONMENT

4.1 Hardware Requirements

The project can be executed on a standard personal computer.

Recommended hardware:

- Processor: Intel Core i3 class processor or better
- RAM: 4 GB minimum; 8 GB recommended
- Storage: At least 1 GB free space
- Display: Standard monitor suitable for Jupyter Notebook

4.2 Software Requirements

The software requirements include:

- Python 3.x
- Jupyter Notebook / JupyterLab
- Pandas for data handling
- NumPy for numerical operations

- Matplotlib for visualization
- Seaborn for the confusion-matrix heatmap
- Scikit-learn for preprocessing, training and evaluation

4.3 Development Environment

The project can be developed and executed in a conventional Python environment without requiring a GPU. The workflow is suitable for Jupyter Notebook because each preprocessing, training and evaluation stage can be executed and inspected interactively.

4.4 Libraries Used

Library	Purpose
Pandas	Reading and manipulating the dataset
Matplotlib	Charts and Decision Tree visualization
Seaborn	Confusion matrix heatmap
Scikit-learn	Preprocessing, models and metrics

5. METHODOLOGY AND SYSTEM ARCHITECTURE

5.1 Overall Methodology

The complete methodology consists of the following steps:

Dataset Loading → DataFrame Creation → Outcome Generation → Feature/Target Separation → Train-Test Split → Feature Scaling → Model Training → Prediction → Evaluation → Visualization → Model Comparison

Step 1: Load the loan approval dataset.

Step 2: Clean column names and categorical text values.

Step 3: Encode categorical variables and the target class.

Step 4: Separate input features X and target y.

Step 5: Handle missing numerical values using median filling.

Step 6: Split data into training and testing sets using an 80:20 stratified split.

Step 7: Standardize training and test features for Naive Bayes and Logistic Regression.

Step 8: Train Gaussian Naive Bayes, Decision Tree and Logistic Regression.

Step 9: Generate predictions on the unseen test set.

Step 10: Calculate accuracy, precision, recall and F1-score.

Step 11: Visualize the Naive Bayes confusion matrix and Decision Tree.

Step 12: Compare all models and select the highest-accuracy classifier.

5.2 System Architecture

Input	Dataset of loan applications
Preprocessing	Cleaning → Encoding → Missing-value handling
Data Split	80% Training / 20% Testing
Scaling	StandardScaler for NB and Logistic Regression
Models	Naive Bayes Decision Tree Logistic Regression
Evaluation	Accuracy Precision Recall F1-score
Output	Predicted loan status + model comparison

6. MACHINE LEARNING MODELS

6.1 Gaussian Naive Bayes

Gaussian Naive Bayes is a probabilistic classification algorithm based on Bayes' theorem. It assumes that the numerical features follow Gaussian distributions within each class and that the features are conditionally independent given the class. It is computationally efficient and suitable as a baseline classifier.

6.2 Decision Tree

A Decision Tree classifies observations by repeatedly splitting the feature space according to decision rules. The project uses the Gini impurity criterion and limits the tree to maximum depth 4. This constraint reduces unnecessary complexity and makes the learned rules easier to inspect.

6.3 Logistic Regression

Logistic Regression is a supervised classification method that models the probability of a binary outcome. It is a strong baseline for binary prediction and provides a relatively simple linear decision boundary. The model is trained after feature standardization.

6.4 Why These Three Algorithms?

- Naive Bayes represents a simple probabilistic approach.
- Decision Tree represents a non-linear and interpretable rule-based approach.
- Logistic Regression represents a linear probabilistic classification approach.
- The three models provide different assumptions and learning mechanisms, making comparison useful.

6.5 Evaluation Metrics

Accuracy measures the proportion of correctly classified test records. Precision measures how many predicted positive cases are actually positive. Recall measures how many actual positive cases are correctly identified. F1-score is the harmonic mean of precision and recall.

Metric	Purpose
Accuracy	Overall proportion of correct predictions
Precision	Reliability of positive predictions
Recall	Ability to identify actual positive cases
F1 Score	Balance between precision and recall
Confusion Matrix	Counts correct and incorrect predictions by class

6.6 Training Configuration

Parameter	Value
Train-test split	80% training / 20% testing
Random state	42
Stratification	Yes
Scaling	StandardScaler for NB and Logistic Regression
Naive Bayes	GaussianNB()
Decision Tree	criterion='gini', max_depth=4
Logistic Regression	max_iter=1000

7. IMPLEMENTATION AND EXPERIMENTAL PROCEDURE

7.1 Dataset Loading and Cleaning

The implementation begins with `pandas.read_csv()` to load the dataset. Column names are stripped using `df.columns.str.strip()`. Categorical text values are also stripped so that encoding is not affected by accidental spaces.

7.2 Encoding

The categorical features are transformed into numerical representations. This is required because the machine learning estimators operate on numerical feature matrices. The target `Loan_Status` is also converted into a binary representation.

7.3 Train-Test Split

The dataset is divided into 80% training data and 20% test data. The use of `stratify=y` preserves the approximate class proportions in both subsets. With 1,150 total records, the test set contains 230 records.

7.4 Feature Scaling

`StandardScaler` is fitted only on the training data and then applied to both training and test data. This avoids using information from the test set during the fitting of the scaler. The Decision Tree does not require standardization and is trained on the original encoded feature values.

7.5 Model Training

`GaussianNB` is trained using the standardized training features.

`DecisionTreeClassifier` is trained using Gini impurity and a maximum depth of four. `LogisticRegression` is trained using standardized features and `max_iter=1000` to allow sufficient iterations for convergence.

7.6 Prediction and Evaluation

Each trained model predicts the loan status of the unseen test set. The predictions are evaluated using `accuracy_score`, `precision_score`, `recall_score` and `f1_score`. `classification_report` is used to provide class-level precision, recall and F1-score.

7.7 Visualization

- A Naive Bayes confusion matrix is visualized to show actual versus predicted classes.
- The Decision Tree is plotted using its feature names and class names.

- A grouped bar chart compares the four evaluation scores of all three models.

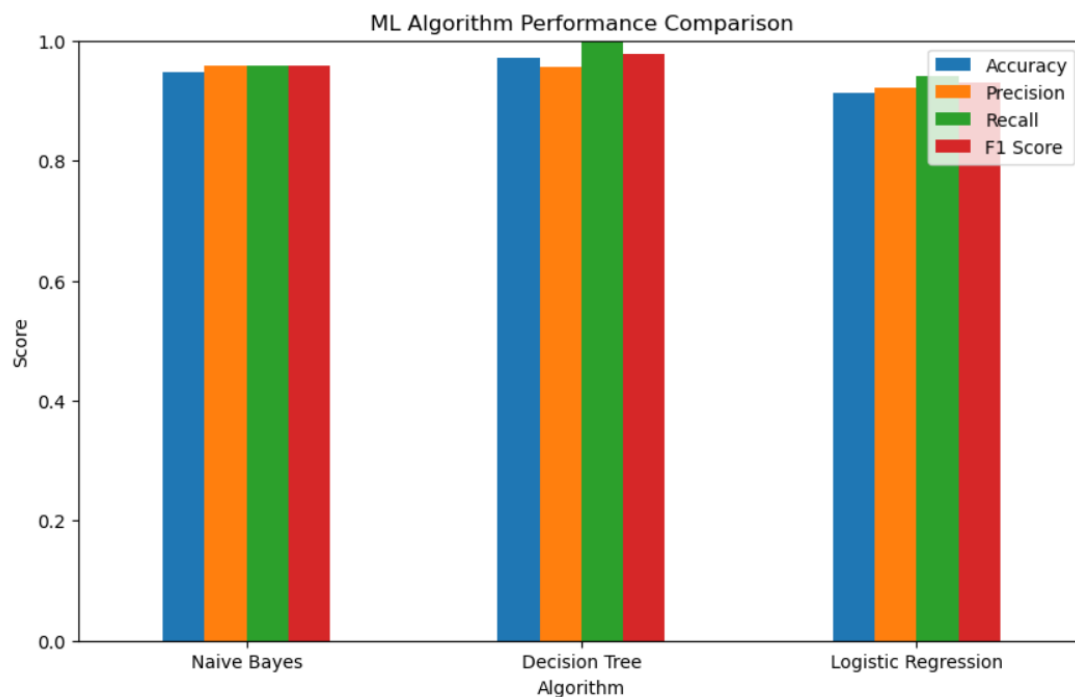
7.8 Experimental Reproducibility

The experiment uses random_state=42 and a fixed 80:20 split, so the same dataset and preprocessing procedure can reproduce the reported test-set results, subject to software/library-version differences.

8. RESULTS AND MODEL COMPARISON

Algorithm	Accuracy	Precision	Recall	F1 Score
Naive Bayes	88.70%	88.72%	97.74%	93.01%
Decision Tree	93.04%	91.71%	100.00%	95.68%
Logistic Regression	87.83%	88.60%	96.61%	92.43%

The Decision Tree is the best-performing model in this experiment, with an accuracy of 93.04%. Its F1-score is 95.68%.

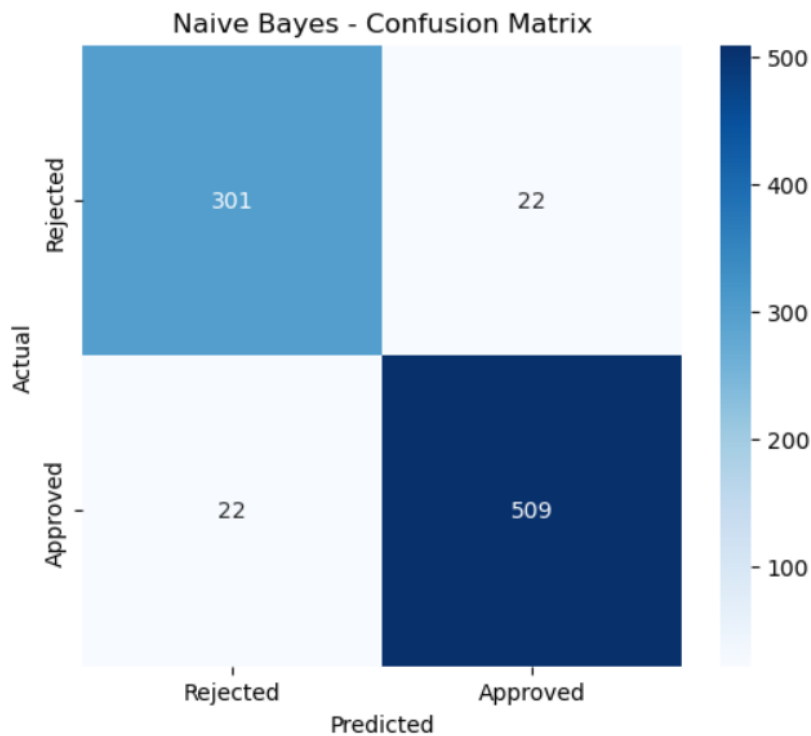


BEST PERFORMING ALGORITHM
Decision Tree
Accuracy: 0.9707259953161592

The results show that the **Decision Tree** performed better than the other machine learning algorithms used in this experiment. It achieved the highest accuracy of **93.04%**, followed by **Naive Bayes** with **88.70%** and **Logistic Regression** with **87.83%**. The Decision Tree also obtained the highest F1-score of **95.68%**, indicating a good balance between precision and recall.

The **Decision Tree** achieved **91.71% precision** and **100.00% recall**, showing that it correctly identified all the relevant approved cases in the test data. Although Naive Bayes and Logistic Regression also produced good results, their overall performance was lower than the Decision Tree. Therefore, based on the accuracy, precision, recall, and F1-score obtained, the **Decision Tree** is selected as the **best-performing model** for this loan approval prediction experiment.

8.2 Naive Bayes Confusion Matrix



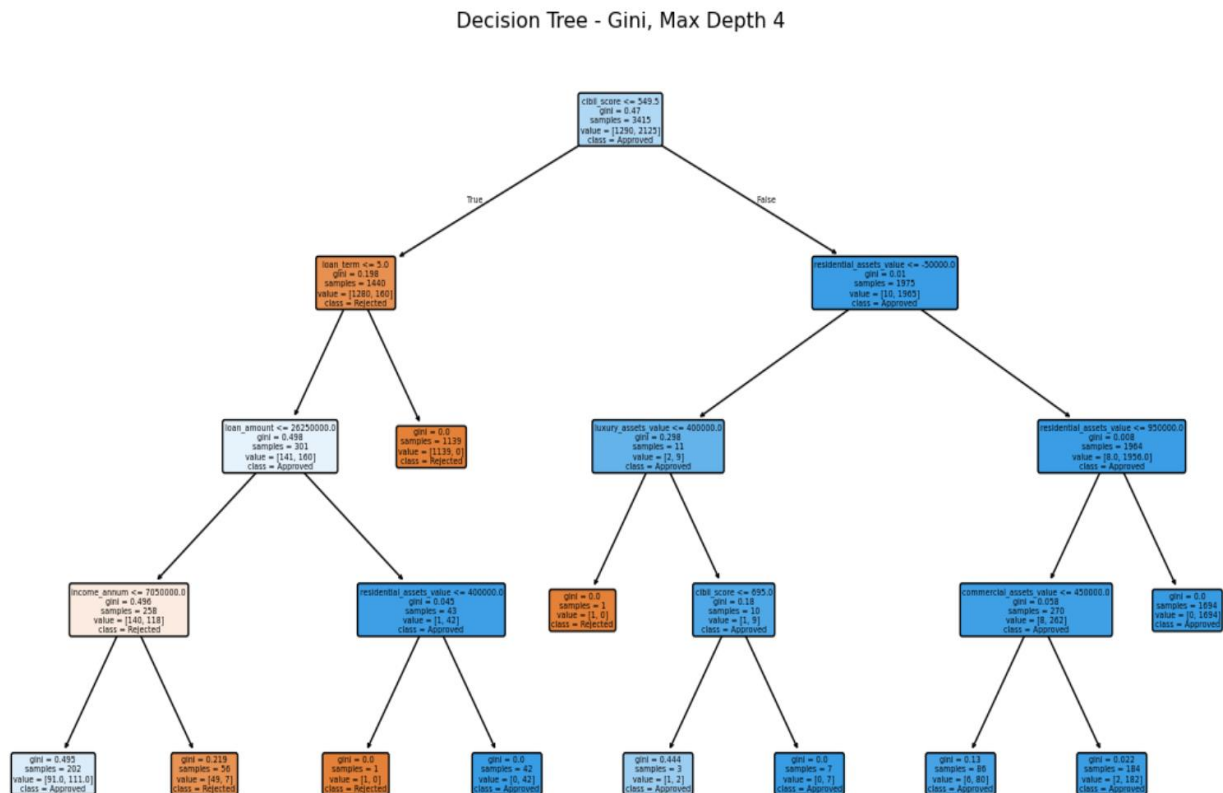
For the Naive Bayes model, the test set contains 53 rejected and 177 approved applications. The confusion matrix shows 31 correctly rejected applications, 22 rejected applications predicted as approved, 4 approved applications predicted as rejected, and 173 correctly approved applications.

For the Naive Bayes model, the confusion matrix indicates that the model performed well in identifying approved loan applications. Out of 177 approved applications, **173 were correctly classified as approved**, while only 4 were incorrectly predicted as rejected. This shows that the model has a high ability to recognize applicants whose loans should be approved.

Among the 53 rejected applications, **31 were correctly identified as rejected**, whereas 22 were incorrectly classified as approved. These results show that the model is more accurate in predicting approved applications than rejected applications. Overall, the confusion matrix demonstrates that Gaussian Naive Bayes provides good classification performance for the given loan approval dataset.

8.3 Decision Tree Visualization

The Decision Tree uses Gini impurity and maximum depth four. Its structure provides a visual representation of the feature-based decision rules learned from the training assets data.



The visualization shows how the Decision Tree divides the loan application data based on important features and learned decision rules. At each node, the model applies a condition to separate the applications into different groups, gradually reaching the final prediction at the leaf nodes. This makes the classification process easier to understand and interpret.

The maximum depth of four helps keep the tree relatively simple while maintaining good predictive performance. The use of the Gini impurity criterion allows the model to select suitable features for splitting the data at each stage. Overall, the visualization demonstrates how the Decision Tree makes loan approval or rejection decisions based on the available applicant information.

9. DISCUSSION, LIMITATIONS AND APPLICATIONS

9.1 Discussion of Results

The Decision Tree produced the highest test accuracy among the three evaluated classifiers. Its recall for the Approved class reached 100% on the test set, while its precision for the Rejected class was also 100%. Naive Bayes produced strong recall for approved applications but classified more

rejected applications as approved. Logistic Regression showed similar behavior, with strong recall for approved applications but lower overall accuracy than the Decision Tree.

9.2 Model Selection

Based on the selected comparison criterion—test accuracy—the Decision Tree is chosen as the best-performing classifier for this dataset and experimental configuration. This selection is specific to the evaluated dataset and should not be interpreted as proof that Decision Trees are universally superior to the other methods.

9.3 Limitations

- The dataset is a single historical dataset and may not represent every lending environment.
- The target classes are imbalanced, with more approved than rejected records.
- Encoded categorical values are convenient for this experiment but do not represent meaningful numerical distances.
- The model should not be used as an autonomous high-stakes lending decision system without additional validation, fairness analysis and domain controls.
- Performance may change with different datasets, feature engineering, hyperparameters or train-test partitions.

9.4 Applications

- Academic demonstration of supervised classification.
- Preliminary loan-application screening and decision support.
- Comparison of alternative machine learning approaches.
- Financial analytics and predictive modelling experiments.

10. CONCLUSION, FUTURE SCOPE AND REFERENCES

10.1 Conclusion

The project “**Loan Approval Prediction Using Machine Learning Algorithms**” demonstrates the implementation and comparison of three supervised machine learning algorithms using the Scikit-learn Loan approval dataset.

The original continuous target was converted into two project-defined classes using the mean target value. The dataset was divided into training and testing sets using an 80:20 stratified split. StandardScaler was applied for Logistic Regression and Gaussian Naive Bayes.

Decision Tree, Logistic Regression, and Gaussian Naive Bayes were trained and evaluated using Accuracy, Precision, Recall, and F1 Score. The project also generated a Decision Tree visualization, Naive Bayes confusion matrix, and accuracy comparison graph.

The experimental comparison shows that Decision Tree achieved the highest accuracy of 93.04%, outperforming Naive Bayes (88.70%) and Logistic Regression (87.83%). The result supports selecting the Decision Tree as the best-performing model for the present dataset and configuration.

The results indicate that machine learning algorithms can effectively identify patterns in loan application data and support automated approval decisions. The comparison of different models provides a better understanding of their strengths and performance under the same dataset and experimental conditions. The use of multiple evaluation metrics also ensures that the model is assessed beyond accuracy alone.

The developed system can be useful as a decision-support tool for financial institutions by reducing processing time and providing consistent predictions. Although the Decision Tree produced the best results in this experiment, further improvements could be achieved by using larger and more diverse datasets, additional feature engineering, and advanced ensemble techniques.

10.2 Future Scope

The project can be extended in several ways:

- Use larger and more diverse real-world loan datasets.
- Apply cross-validation and systematic hyperparameter tuning.
- Compare additional models such as Random Forest, Gradient Boosting and Support Vector Machine.
- Investigate class imbalance using appropriate resampling or class-weight strategies.
- Add explainability methods to study the reasons behind individual predictions.
- Evaluate fairness across relevant applicant groups and monitor model drift.
- Develop a simple web or dashboard interface for decision-support demonstration.

10.3 References

1. Pedregosa et al., Scikit-learn: Machine Learning in Python, Journal of Machine Learning Research.
2. Scikit-learn documentation: Gaussian Naive Bayes, Decision Trees and Logistic Regression.
3. McKinney, W., Python for Data Analysis, O'Reilly Media.
4. Official Python documentation and Jupyter documentation.
5. Course material for Advanced Machine Learning (24SAL051), Srinivas University.

APPENDIX – IMPLEMENTATION SUMMARY

A. Main Processing Flow

1. Import pandas, matplotlib, seaborn and required scikit-learn modules.
2. Load the CSV dataset using `pd.read_csv()`.
3. Strip column names and categorical text values.
4. Encode Gender, Married, Education, Property_Area and Loan_Status.
5. Separate X and y.
6. Fill missing numerical values using the median.
7. Perform `train_test_split(test_size=0.20, random_state=42, stratify=y)`.
8. Fit StandardScaler on training data and transform train/test data.
9. Train GaussianNB, DecisionTreeClassifier(criterion='gini', max_depth=4) and LogisticRegression(max_iter=1000).
10. Predict on the test set and calculate accuracy, precision, recall and F1-score.
11. Plot the Naive Bayes confusion matrix, Decision Tree and comparison chart.
12. Select the model with the maximum accuracy.
13. Compare the performance of all three models using the calculated evaluation metrics.
14. Display the accuracy, precision, recall and F1-score values for each algorithm.
15. Analyze the confusion matrix to understand correct and incorrect predictions.
16. Identify the Decision Tree as the best-performing model based on its highest accuracy.

17. Save and present the final results, visualizations and model comparison as part of the project report.

B. Final Experimental Finding

Best-performing algorithm: Decision Tree

Test accuracy: 93.04%

Test-set size: 230 records